SQL LAB EXAM

One-Page Cheat Sheet

Read this the morning of the exam. Templates + word-to-SQL dictionary + error fixes.

The 5-step method (any table)

1. Read the schema. Mark PKs and FKs. The table with 2 FKs is the middle / child table.
2. Split the question at every full stop. Each sentence = one SQL statement.
3. Spot the keyword and pick the recipe below.
4. Fill the template with the new table / column names.
5. Insert 2–3 sample rows yourself, run it, and show `SELECT * FROM ...;`

Golden rules

- Every statement ends with `;`
- Text and dates in single quotes: `'Chennai'` `'2024-05-10'`
- Numbers without quotes: `8.5` `75000`
- Create parent tables first, child (FK) tables after
- Insert parent rows before child rows
- ID like C1 / F1 → `VARCHAR(10)`, not INT
- Money → `DECIMAL(10,2)`, CGPA → `DECIMAL(4,2)`, rating → `DECIMAL(2,1)`

Question words → SQL

"for each / per"	`GROUP BY col`

"having more than N"	`HAVING COUNT(*) > N`
"greater than average"	`WHERE c > (SELECT AVG(c) FROM t)`
"equal to maximum"	`WHERE c = (SELECT MAX(c) FROM t)`
"at least one"	`WHERE id IN (SELECT id FROM other)`
"never / have not"	`WHERE id NOT IN (SELECT id FROM other)`
"of that same X"	`correlated: ... WHERE b2.X = b1.X`
"include those who have not"	`LEFT JOIN (keep-all table first)`
"every A and every B, blanks"	`LEFT JOIN ... UNION ... RIGHT JOIN`
"all possible combinations"	`CROSS JOIN`
"pairs in the same ..."	`SELF JOIN with e1.id < e2.id`
"descending / highest first"	`ORDER BY c DESC`
"count / total / average"	`COUNT(*) / SUM(c) / AVG(c)`
"highest / lowest"	`MAX(c) / MIN(c)`
"increase by 10%"	`SET c = c * 1.10`
"add ₹500"	`SET c = c + 500`

Create table + constraints

```
CREATE DATABASE db;
USE db;

CREATE TABLE T (
  id    INT PRIMARY KEY,
  name  VARCHAR(50) NOT NULL UNIQUE,
  sal   DECIMAL(10,2) CHECK (sal > 20000),
  cgpa  DECIMAL(4,2) CHECK (cgpa BETWEEN 0 AND
10),
  price DECIMAL(6,2) DEFAULT 150,
  pid   INT,
  FOREIGN KEY (pid) REFERENCES P(pid)
      ON DELETE CASCADE        -- or ON UPDATE
CASCADE
);
ALTER TABLE T ADD CONSTRAINT fk1
  FOREIGN KEY (pid) REFERENCES P(pid);
DESC T;
```

Insert / update / delete

```
INSERT INTO T VALUES (1, 'A', 30000, 8.5, 200,
5);
INSERT INTO T (id, name, sal, pid)
  VALUES (2, 'B', 40000, 5);        -- DEFAULT
used
UPDATE T SET sal = 75000 WHERE name = 'Meena';
UPDATE T SET sal = sal * 1.10;      -- +10%
DELETE FROM T WHERE id = 2;
-- FK insert fails (ERROR 1452): parent value
-- does not exist = referential integrity
```

Select order

```
SELECT   cols
FROM     t
WHERE    row condition
GROUP BY col
HAVING   COUNT(*) > 1      -- condition with
COUNT/SUM/AVG
ORDER BY col DESC;
```

Joins

```
-- 2 tables
SELECT e.EmpName, d.DeptName
FROM Employee e JOIN Department d ON e.DeptID =
d.DeptID;
-- 3 tables: start from the middle table
SELECT p.PatientName, d.DoctorName
FROM Appointment a
JOIN Patient p ON a.PatientID = p.PatientID
JOIN Doctor  d ON a.DoctorID  = d.DoctorID;
-- LEFT / FULL OUTER / CROSS / SELF
... FROM A a LEFT JOIN B b ON a.id = b.id;
SELECT .. LEFT JOIN .. UNION SELECT .. RIGHT
JOIN ..;
SELECT * FROM A CROSS JOIN B;
SELECT e1.n, e2.n FROM E e1 JOIN E e2
  ON e1.d = e2.d AND e1.id < e2.id;
```

Transactions

```
SET autocommit = 0;
START TRANSACTION; ...changes...; COMMIT;
```

```
START TRANSACTION; ...change...;
SELECT * FROM t; ROLLBACK; SELECT * FROM t;

START TRANSACTION;
A;  SAVEPOINT sp1;  B;
ROLLBACK TO sp1;    -- B undone, A kept
COMMIT;
```

Grant / revoke

```
CREATE USER 'u'@'localhost' IDENTIFIED BY
'pass123';
GRANT SELECT, INSERT ON db.T TO 'u'@'localhost';
REVOKE INSERT ON db.T FROM 'u'@'localhost';
GRANT ALL PRIVILEGES ON db.T TO 'u'@'localhost';
SHOW GRANTS FOR 'u'@'localhost';
```

View + index

```
CREATE VIEW V AS
SELECT ... FROM ... JOIN ... ON ...;
SELECT * FROM V;
CREATE INDEX idx_t_col ON T(col);
SHOW INDEX FROM T;
```

Procedure / function

```
DELIMITER //
CREATE PROCEDURE p(IN x INT)
BEGIN
  SELECT * FROM T WHERE id = x;
END //
CREATE FUNCTION f(a INT, b INT)
```

```
RETURNS INT DETERMINISTIC
BEGIN
  RETURN a + b;
END //
DELIMITER ;
CALL p(1);   SELECT f(2, 3);
-- IF m >= 40 THEN RETURN 'PASS'; ELSE RETURN
'FAIL'; END IF;
```

Trigger

```
DELIMITER //
CREATE TRIGGER trg AFTER INSERT ON T FOR EACH
ROW
BEGIN
  INSERT INTO T_Log(id, action, at)
  VALUES (NEW.id, 'INSERT', NOW());
END //
DELIMITER ;
-- INSERT: NEW only · UPDATE: OLD + NEW ·
DELETE: OLD only
-- Test: do an INSERT, then SELECT * FROM T_Log;
```

Error → fix

1452 FK fails	Insert the parent row first
1062 Duplicate entry	PK / UNIQUE value already used
3819 / 4025 CHECK	Value breaks the CHECK rule
1064 Syntax	Missing comma, quote, bracket or `;`
1050 Table exists	`DROP TABLE IF EXISTS t;` (child before parent)

ROLLBACK did nothing	Forgot `SET autocommit = 0;` / `START TRANSACTION;`